

best-effort kernels at microsecond-scale. Second, since DNN inference kernels have varied parallelism and predictable latency, REEF proposes a dynamic kernel padding mechanism that dynamically pads the real-time kernel with appropriate best-effort kernels to fully utilize the GPU with negligible overhead. Evaluation using a new DNN inference serving benchmark (DISB) with diverse workloads and a real-world trace on both NVIDIA and AMD GPUs shows that REEF only incurs less than 5% overhead in end-to-end latency for real-time tasks but increases the overall throughput by up to 1.53 $\times$, compared to scheduling tasks sequentially. To demonstrate the practical benefits of our approach, we compare REEF with Triton, a widely-adopted production-level serving system. Our evaluation shows that REEF outperforms Triton by 1.12 $\times$ to 5.20 $\times$ in end-to-end latency for real-time tasks, while maintaining comparable throughput.

CCS Concepts: • **Software and its engineering** → **Scheduling**; • **Computing methodologies** → **Concurrent computing methodologies**;

Additional Key Words and Phrases: GPU scheduling, kernel preemption, kernel padding, DNN inference

ACM Reference Format:

Mingcong Han, Rong Chen, Weihang Shen, Hanze Zhang, Jinrong Yang, and Haibo Chen. 2025. Real-time, Work-conserving GPU Scheduling for Concurrent DNN Inference. *ACM Trans. Comput. Syst.* 44, 1, Article 1 (November 2025), 42 pages. <https://doi.org/10.1145/3768622>

1 Introduction

Deep Neural Network (DNN) inference has been widely adopted by modern intelligent applications, such as autonomous driving [2, 46, 54, 108], virtual reality [78, 112], speech/image recognition [42, 102], and healthcare [23, 30], just to name a few. Many of them demand real-time inference serving in mission-critical tasks, where GPUs have emerged as a popular accelerator to serve DNN inferences [17, 43, 62, 120].

Although the low-latency demand of DNN inferences can be fulfilled by dedicating the whole GPU to sequentially serve requests from a single DNN application [12, 108, 121], it is hard to fully exploit the massive parallelism of the GPU [62]. Hence, it is a common practice to share a GPU among multiple applications with different timing constraints in emerging intelligent systems [54, 105, 107], which can greatly improve overall throughput, as shown in Figure 1(a). For example, autonomous vehicles use DNNs to recognize obstacles and traffic lights [11, 36, 81], which are latency-critical tasks (called *real-time* tasks in this article). Meanwhile, other tasks with no hard real-time requirement [105] (called *best-effort* tasks in this article), such as monitoring human driver's emotion and fatigue, are also served within the GPU using DNNs [23, 65, 113].

Typically, DNN inferences have two potentially conflicting goals for GPU scheduling. First, the real-time tasks should be treated as first-class citizens on the GPU without interference from other tasks to achieve *low end-to-end latency*. Second, both real-time tasks and best-effort tasks should be served concurrently on the GPU to achieve high overall throughput (*work-conserving*).

State-of-the-art GPU libraries (e.g., CUDA [71] and ROCm [3]) commonly provide multiple GPU streams (e.g., CUDA Streams [83]) to concurrently execute multiple tasks on the same GPU. However, as shown in Figure 1(b), although the end-to-end inference latency of real-time tasks is low (about 4 ms) and stable when monopolizing the GPU, the tail latency of real-time tasks significantly increases by over an order of magnitude (close to 50 ms) when running concurrently with best-effort tasks. This, unfortunately, is unacceptable for real-time scenarios [116].

Similar to operating systems using preemptive scheduling to provide real-time guarantees, an intuitive approach is to provide preemption for GPU scheduling, which is unfortunately missing in commodity GPUs [96]. Prior work [14, 104, 122] proposed a wait-based approach to passively waiting until the completion of running blocks, which may cause a preemption delay of several milliseconds. Although it may be sufficient for traditional GPU workloads, this approach is still far

to construct a template of padded kernels by using function pointers. Furthermore, to eliminate the overhead of indirect function calls on the GPU, REEF introduces proxy kernels to address register allocation problem and avoid unnecessary context saving at runtime. Therefore, REEF can concurrently execute the real-time task with best-effort tasks at the expense of negligible performance and memory overhead (less than 1% and about 10 KB).

We initially implemented REEF on AMD GPUs (e.g., Radeon Instinct MI50) and subsequently ported it to NVIDIA GPUs (e.g., GV100). REEF extends Apache TVM [98], a deep learning compiler, to generate customized GPU kernels for DNN inferences, implementing reset-based preemption and dynamic kernel padding mechanisms. Our reset-based preemption scheme has influenced XSched [91], a preemptive scheduling framework for diverse XPU, enabling efficient preemption of both pending and running kernels.¹ We evaluate REEF using a new **DNN Inference Serving Benchmark (DISB)** with diverse workloads and models, as well as a real-world trace from Apollo [8] (an open autonomous driving platform). The evaluation conducted on AMD GPUs shows that REEF only incurs less than 5% end-to-end latency overhead for real-time tasks while achieves up to a 1.53× throughput improvement compared to scheduling tasks sequentially. Our approach further reduces the preemption latency by over one order of magnitude against the state-of-the-art, less than 40 microseconds for all models on both AMD and NVIDIA GPUs. To demonstrate the feasibility of our approaches, we conducted evaluations of REEF on an NVIDIA GPU using inference engine with closed-source GPU kernels (e.g., TensorRT [68]). REEF outperforms Triton [22], a production-level serving system, by 1.12× to 5.20× in end-to-end latency for real-time tasks, while maintaining comparable throughput.

Contributions. We summarize our contributions as follows.

- An in-depth understanding on the characteristics of GPU-accelerated DNN inferences, such as idempotence and the issues of state-of-the-art GPU scheduling schemes (Section 2).
- A novel reset-based preemption scheme that launches a real-time kernel on the GPU in a few microseconds, regardless of the number of preempted kernels (Section 4).
- An elegant mechanism that can dynamically pad the real-time kernel with best-effort kernels to fully exploit the massive parallelism of the GPU (Section 5).
- A prototype implementation for both AMD and NVIDIA GPUs and an evaluation that demonstrates the advantage and efficacy of REEF over state-of-the-art approaches (Section 6).

The source code of REEF is publicly available at <https://github.com/SJTU-IPADS/reef>. The DISB framework can be obtained separately from <https://github.com/SJTU-IPADS/dish>.

2 Background and Motivation

2.1 Characterizing GPU-Accelerated DNN Inference

DNN comprises multiple instances of versatile layers, such as convolutional, pooling and fully-connected layers. GPUs have been widely exploited to accelerate DNN inference serving [24, 35, 87]. To serve inference requests on GPUs, the pre-trained DNN model (e.g., ResNet [40]) is loaded into GPU memory ahead of time. Figure 2 outlines the implementation of GPU-accelerated DNN inference. For each arriving request, all kernels of the DNN model are executed in turn with the input, and the resulting output is returned to the DNN application.

DNN inference is now used by both *real-time* (RT) tasks, such as obstacle and traffic lights recognition [11, 81], and *best-effort* (BE) tasks, such as emotion and fatigue monitoring [23, 65, 113]. The real-time tasks are latency-critical, because violating the end-to-end latency requirement may cause system failures or even safety problems. In addition, such requests are usually issued

¹XSched is publicly available at <https://github.com/XpuOS/xsched>

```

# device codes
__global__ void conv_relu(in, weight, out):
1  sum = 0;
2  for i in range(0,3)
3      for j in range(0,3)
4          sum += in[..] * weight[..]
5  out[..] = ReLU(sum)

__global__ void dense(in, weight, bias, out):
6  sum = 0;
7  for i in range(0,512)
8      sum += in[..] * weight[..]
9  out[..] = sum + bias[..]

# host codes
void inference(...):
10 memcpH2D(in, in_host, in_sz) # copy in to GPU
11 conv_relu <<<dim(32), ...>>> (in, .., buf_conv)
12 ... # launch other kernels
13 pooling <<<dim(64), ...>>> (.., buf_pool)
14 dense <<<dim(10), ...>>> (buf_pool, .., buf_dense)
15 softmax <<<dim(1), ...>>> (buf_dense, .., out)
16 memcpD2H(out_host, out, out_sz) # copy out to CPU

```

Fig. 2. An example of DNN inference using a ResNet-like model.

Table 1. Number of Idempotent GPU Kernels and Total GPU Kernels for DNN Models Evaluated in Section 6 Across Different Inference frameworks (TVM [98], TensorRT [68], and PyTorch [77])

Model	#Idem Kernels / #Total Kernels			Execution Time
	TVM	TensorRT	PyTorch	
ResNet-152 [40] (RNET)	207/207	67/103	145/211	9.1
VGG-19 [94] (VGG)	71/ 71	30/ 30	45/ 81	3.7
Yolov3 [82] (YOLO)	144/144	153/175	569/572	10.0
Bert-Base [28] (BERT)	393/393	161/173	165/238	8.1
GPT-2 [80] (GPT2)	440/440	136/148	258/294	13.2

Their execution times (in milliseconds) are measured on an AMD Radeon Instinct MI50 GPU using TVM.

periodically at various frequencies by input sensors (e.g., camera and LiDAR [8, 54]). On the contrary, the best-effort tasks have no hard timing requirement, but are repetitively executed in the background.

Idempotence. The GPU-accelerated DNN model for inference tasks consists of a sequence of kernels, which implement one or several DNN layers. We observe that most GPU kernels in DNN models are *idempotent* as they primarily perform dense linear algebra computations without side effects. Through our analysis of five DNN models across three major inference frameworks, we found that all kernels generated by TVM [98] are idempotent, while 1182 out of 1396 kernels from PyTorch [77] and 547 out of 629 kernels from TensorRT [68] are idempotent, as detailed in Table 1. The idempotence property ensures that a kernel will always produce identical output given the same input, regardless of whether it has been retried or not. This characteristic enables efficient preemption of DNN inference tasks by simply killing the running kernel and restoring execution by re-starting it. For the small fraction of non-idempotent kernels, we note that each kernel in a DNN model only depends on outputs from previous kernels and static arguments (e.g., weights) as inputs. For instance, the dense kernel in Figure 2 takes the output of the pooling kernel and a

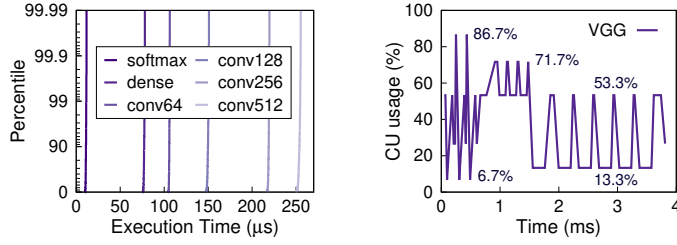


Fig. 3. (a) The CDF of execution time (in μ s) for several typical kernels in VGG, and (b) the timeline of CU usage during VGG execution on a GPU with 60 compute units (CUs). The execution time of GPU kernels in VGG ranges from 10 μ s to 365 μ s (see Figure 13).

weight matrix as inputs. This dependency structure allows us to safely restore a preempted DNN inference task by re-executing it from the previous kernel, even if the preemption occurred during a non-idempotent kernel's execution.

Massive kernels. Unlike traditional GPU applications that only contain a few kernels (e.g., at most 14 kernels in Rodinia [13]), it is common to see hundreds of kernels in modern DNN models (see Table 1). In response, large amounts of kernels—usually hundreds or more—would be submitted in advance to hide the lengthy kernel launching time. Furthermore, to fully exploit the GPU, the serving system may concurrently execute multiple kernels from different inference tasks using the same or different DNN models. Therefore, the performance penalty of preempting the GPU would be significant (a few milliseconds) and even comparable to the execution time of hundreds of kernels.

Latency predictability. We observe that the execution time of GPU kernels in DNN inferences is deterministic and predictable when running individually on the GPU (no interference). The reasons are two-fold. First, the kernel is mostly linear algebra computations such as matrix multiplication and convolution, which contains neither conditional branches nor inconstant loops. Second, all kernel arguments (e.g., input and weights) and the output are fixed-size arrays. Therefore, the execution time of such kernels is independent of the input of inference request and can be measured and accurately predicted in advance. In practice, we observe that the variance in kernel execution time of DNN models is typically only a few microseconds (see Figure 3(a)). This is also confirmed in recent literature [7, 35, 62].

Varied parallelism. The GPU kernels in DNN inferences usually exhibit completely different parallelism due to varied input scales. For example, as shown in Figure 2, the pooling kernel uses 64 thread blocks, while the softmax kernel just uses 1 thread block. Consequently, the computational demand for DNN inferences, namely the number of compute units (CUs), is ever-changing during the execution. As an example, Figure 3(b) shows the CU usage during VGG execution varies between 6.7% and 86.7%. Therefore, to efficiently exploit the GPU, it is indispensable to leverage a dynamic mechanism to select and execute multiple kernels from different DNN inference tasks at runtime.

2.2 State-of-the-Art GPU Scheduling

As stated before, DNN inference serving system relies on GPU scheduling to meet two potentially conflicting performance goals: low latency and work conserving. Although GPU scheduling has been widely studied in the HPC community [1, 10, 14, 15, 34, 58, 104, 111, 119], the unique characteristics of DNN inferences and the two performance goals introduce new challenges for GPU scheduling. We review the state-of-the-art schemes of GPU scheduling and discuss the performance issues when serving various DNN inferences through a brief example, as shown in Figure 4.

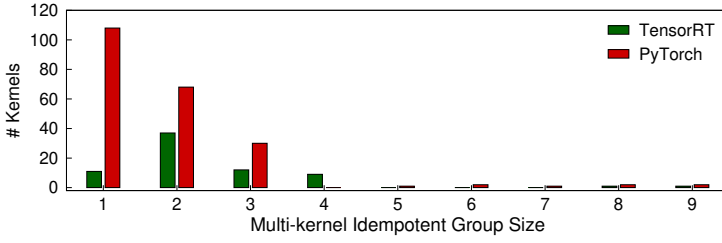


Fig. 10. The number of kernels in the minimal idempotent group containing non-idempotent kernels for DNN models in Table 1.

Restoring idempotent kernel group. After identifying the minimal idempotent group for each kernel, REEF can efficiently restore preempted tasks by re-executing the appropriate kernel group. When restoring a preempted task, the preemption module first reads the global execution index to determine which kernel was interrupted. If the interrupted kernel is idempotent, REEF simply re-executes that kernel and continues with subsequent kernels. For non-idempotent kernels, REEF looks up its pre-computed minimal idempotent group and re-executes all kernels in that group, starting from the first kernel in the group.

4.5 Reset-Based Preemption on Open-Source GPUs

Our queue abstractions were originally inspired by the design of ROCm [3], an open-source GPU runtime for AMD GPUs. ROCm has exactly the data structure of host queue and DQ as described in Section 4.1. Therefore, REEF implements the preemption module on AMD GPUs by directly modifying its implementation of host queue and DQ in ROCm, and adding three new APIs to HIP [86].

Queue eviction. REEF adds two new APIs to the HIP for managing host queues and DQs: (1) `hip_reset_hq`, which resets a host queue and moves the untransmitted kernels to the GC thread; (2) `hip_set_stream_cap`, which limits the capacity of the DQ used by a GPU stream.

Kernel killing. REEF retrofits the GPU reset function of Linux AMD GPU driver [84], which instructs the GPU command processor to kill all running kernels on the CUs. We expose this function to the preemption module via `ioctl` system call, and add a new API `hip_reset_kern` to the HIP to interrupt and kill all running kernels. Our experiments show that the kernel killing function can terminate all running kernels in less than $3\mu\text{s}$ on our testbed.

4.6 Reset-Based Preemption on Closed-Source GPUs

The closed-source GPU runtime, such as CUDA [71] of NVIDIA GPUs, poses two main challenges for implementing reset-based preemption. First, the CUDA runtime does not expose the queues to the user. Even worse, prior research also discovered that CUDA kernels are launched to the DQs, without buffering in a host queue [12]. As a result, our techniques for manipulating the queues cannot be directly applied to NVIDIA GPUs outside of the CUDA runtime. Second, the CUDA runtime also does not provide an API to reset the CUs, making the kernel killing mechanism also difficult to implement. We have implemented REEF for NVIDIA GPUs, which tackles these challenges with the following techniques.

Queue eviction. To address the first challenge, REEFnv first wraps each CUDA stream, into a *virtual* host queue (vHQs), which intercepts and buffers all launched kernels. Similar to the (physical) HQ, each vHQ also has a background thread to transmit buffered kernels asynchronously to the CUDA runtime. After that, REEFnv treats the whole GPU runtime as several DQs (one for each GPU stream), such that REEFnv can easily reset vHQs to evict buffered kernels, instead of resetting HQs

Table 2. DISB Workload Description

DISB	A	B	C	D	E
Num. of RT clients	1/VGG	1/VGG	1/VGG	5/ALL	5/ALL
Avg. throughput per RT client	50% [U]	100%	50% [U]	10% [U]	10% [P]
Num. of BE clients	1/RNET	1/RNET	5/ALL	5/ALL	5/ALL

“#/model” denotes the number of clients and their DNN models. “[U/P]” denotes the arrival distribution (i.e., Uniform or Poisson).

kernel binary to modify the register count of each proxy kernel, which is a feasible approach to generate proxy kernels for NVIDIA GPUs.

Handling constant memory. NVIDIA GPUs offer a distinct memory segment known as constant memory, characterized by its read-only nature and caching capabilities. The NVIDIA GPU compiler, namely `nvcc`, has a unique feature of utilizing constant memory to hold literal constants, which is not found in AMD GPUs. Kernels access this memory through relative addressing, involving both the relative bank number and the offset within the bank. Notably, the constant memory address space may be shared between the real-time kernel and the best-effort kernel when they are executed via dynamic kernel padding, potentially leading to contention issues. Fortunately, only a few kernels (2 kernels in Table 1) are observed to use constant memory, so we disable dynamic kernel padding for the kernels that use constant memory, which is a simple yet effective approach to address the contention issue.

6 Evaluation

6.1 Implementation

We have implemented REEF on both AMD and NVIDIA GPUs with about 8.7k lines of C++ code. For AMD GPUs, REEF extends AMD ROCm [3] to support the reset-based preemption technique. REEF uses TVM [17] to generate DNN inference kernels, and extends it to support the dynamic kernel padding mechanisms. To demonstrate the generality and the extensibility of REEF, we also adapted REEF to support serving inference requests using TensorRT [68] and PyTorch [77].

6.2 Experimental Setup

Testbed. The experiments were mainly conducted on a GPU server that consists of one Intel Core i7-10700 CPU (total 8 cores), 32 GB of DRAM, and one AMD Radeon Instinct MI50 GPU (60 CUs and 16 GB of memory). The software environment of the server was configured with ROCm 4.3.0 [3], Apache TVM [98] 0.14.0, and Ubuntu 20.04. The hardware platform resembles the computational resources of autonomous vehicles [4, 97]. We further evaluate REEFnv on a NVIDIA GV100 GPU (80 SMs and 32 GB of memory) to demonstrate the generality of our approach, using the same server with CUDA 11.4 [71], PyTorch 1.12.0 [77], and TensorRT 8.5.1 [68] installed.

Workloads. Inspired by YCSB [20, 21], we build a new DNN inference serving benchmark (DISB) that contains a suite of tools and five workloads: (A) low load, (B) high RT load, (C) high BE load, (D) multi-RT load, and (E) random load, summarized in Table 2. The real-time (RT) clients in DISB A–D uniformly send inference requests at a given frequency, which simulates real-time DNN applications in autonomous driving (e.g., obstacle recognition with cameras [8]), while the clients in DISB E send requests with a Poisson arrival distribution, which simulates event-driven real-time DNN applications (e.g., speech recognition [42, 102]). On the other hand, the closed-loop best-effort (BE) client continuously issues inference requests, which simulates a contention load on the GPU (e.g., driver monitoring). The request frequency of each RT client is set based on a relative throughput

target, e.g., 50% for DISB A and C, meaning that each RT client sends requests at a frequency of 50% of the maximum throughput that can be achieved by the GPU. For DISB B, the RT client sends requests at the maximum throughput, and for DISB D and E, each RT client sends requests at 10% of the maximum throughput.

Five representative DNN models are deployed, including ResNet-152 [40] (RNET), VGG-19 [94] (VGG), Yolov3 [82] (YOLO), Bert-Base [28] (BERT), and GPT-2 [80] (GPT2). For BERT and GPT2, we use a 128-token sequence length input, and for GPT2, the model only executes one forward pass (i.e., prefill) for each request. Each DISB client always submits inference requests for a certain DNN model. Specifically, VGG is used by DISB A–C for their RT clients, and RNET is used by DISB A and B for their BE clients. Workloads with 5 RT/BE clients deploy all five DNN models in their clients separately, which simulates multiple DNN applications in a single scenario (e.g., autonomous vehicles [8, 54]).

Furthermore, we use a real-world trace from an open autonomous driving platform (i.e., Apollo [8]) as the real-time workload, which provides a realistic arrival distribution of real-time tasks in autonomous driving. The trace was collected from the logs of the perception module [5] when running Apollo with SVL simulator [56, 88], and we selected the closest DNN models in terms of execution time from the above five models for the inference requests. Meanwhile, the same best-effort workload as DISB C–E is used, where five clients continuously issue different DNN inference requests.

Currently, each workload in DISB represents a particular mix of real-time and best-effort DNN inference tasks, the number of clients, and request frequency, which focuses on a particular point in the performance space. Users can further extend DISB with new workloads, or even some production traces from specific applications, to model more different scenarios.

Comparing targets. We compare REEF with typical scheduling approaches. **SEQ** sequentially runs each DNN inference task on the GPU with passive task preemption, which is adopted by Clockwork [35]. Specifically, when there are multiple tasks waiting in the queue, it prioritizes real-time tasks, but still needs to wait for the completion of launched best-effort tasks. **GPUStreams** runs both real-time and best-effort tasks simultaneously on the same GPU through multiple GPU streams, which is adopted by Triton [22]. As a reference, we further provide **RT-Only**, which represents the optimal end-to-end latency for real-time tasks, as it dedicates the GPU to real-time tasks.⁵

Metrics. We use two key metrics to evaluate the performance of different approaches: (1) normalized end-to-end latency for real-time requests and (2) normalized throughput for all clients. For real-time requests, we normalize each request's end-to-end latency by its model's standalone execution time. The standalone execution time is pre-measured offline by taking the average execution time over 1000 runs. For throughput, we normalize each client's throughput to the maximum throughput achievable by its model when running alone on the GPU. These normalized metrics allow us to fairly compare performance across different DNN models and scheduling approaches.

6.3 Overall Performance

We first compare the normalized end-to-end latency of real-time tasks and the normalized overall throughput of all clients using DISB workloads and a real-world trace. The experiments are conducted on both AMD MI50 and NVIDIA GV100 GPUs, and the results are shown in Figures 15 and 16.

Single-RT Single-BE Client (DISB A and B). For workloads with a single real-time and best-effort client, SEQ significantly increases real-time task latency by 2.87× and 3.66× on AMD MI50 GPU,

⁵In this case, additional GPUs are dedicated to best-effort tasks, which also results in extra cost and energy consumption, as well as low GPU utilization.

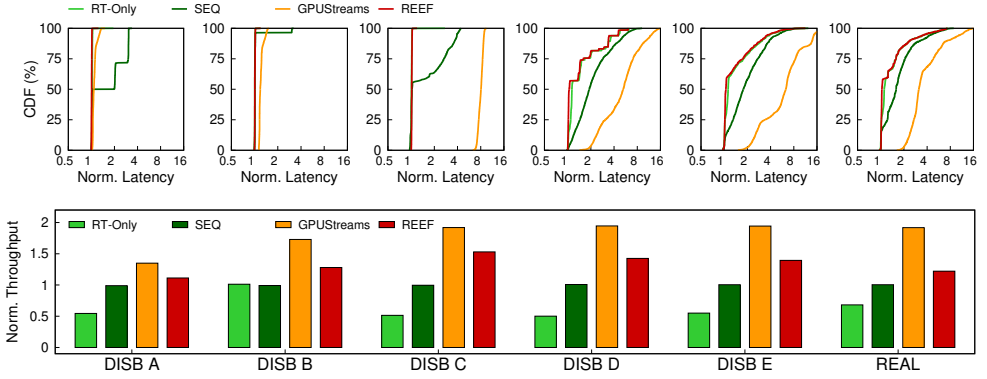


Fig. 15. Comparison of (a) normalized end-to-end latency for RT tasks, and (b) normalized overall throughput of all tasks using various scheduling approaches. **Testbed:** One AMD MI50 GPU.

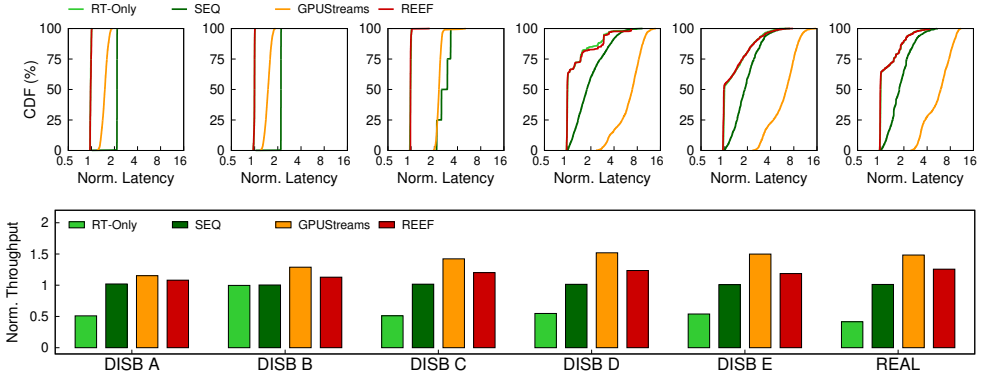


Fig. 16. Comparison of (a) normalized end-to-end latency for RT tasks, and (b) normalized overall throughput of all tasks using various scheduling approaches. **Testbed:** One NVIDIA GV100 GPU.

and $2.16\times$ and $2.19\times$ on NVIDIA GV100 GPU. This substantial latency increase stems from SEQ's inability to preempt running best-effort tasks, forcing real-time tasks to wait for their completion. The waiting time is entirely dependent on the remaining execution time of the best-effort task. In contrast, GPUStreams shows more moderate latency increases of $1.1\times$ and $1.21\times$ on AMD MI50, and $1.46\times$ and $1.51\times$ on NVIDIA GV100. While GPUStreams can execute real-time tasks without waiting for best-effort tasks to complete, the concurrent execution through multiple streams introduces GPU resource contention, leading to interference with real-time task performance. The degree of performance degradation varies based on the level of resource competition from best-effort tasks — less severe on AMD GPUs due to lower CU utilization by best-effort tasks, and more pronounced on NVIDIA GPUs. Notably, REEF maintains near-optimal real-time task performance with only a 0.5% latency increase.

From a throughput perspective, SEQ's single-stream execution limits its throughput to $1.0\times$ regardless of workload. GPUStreams achieves higher throughput of $1.34\times$ and $1.72\times$ on AMD MI50, and $1.15\times$ and $1.28\times$ on NVIDIA GV100. REEF leverages dynamic kernel padding to execute best-effort tasks alongside real-time tasks, achieving throughput improvements of $1.12\times$ and $1.28\times$ on AMD MI50, and $1.08\times$ and $1.12\times$ on NVIDIA GV100. Notably, even in DISB B where real-time tasks saturate 100% of GPU resources, REEF successfully prevents best-effort task starvation.

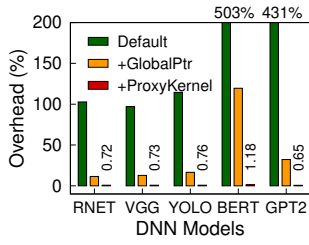


Fig. 25. Comparison of execution time overhead for padded kernels using various optimizations.

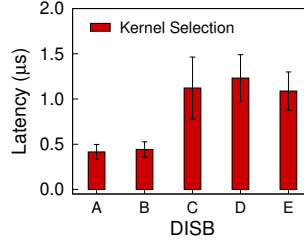


Fig. 26. Execution time of kernel selection for DISB A–E.

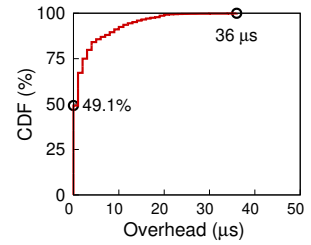


Fig. 27. CDF of execution time overhead for RT kernels using dynamic kernel padding.

higher throughput improvements when executing alongside the same real-time task. This factor also affects GPUstreams' performance in a similar way. Second, the relative relationship between kernel execution times of real-time and best-effort tasks determines the opportunity for kernel padding. When real-time tasks have relatively longer kernel execution times compared to best-effort tasks, there are more opportunities to pad them together. For example, VGG's kernels have relatively long execution times (see Figure 13), so when used as a real-time task, it allows for throughput improvements across all best-effort tasks. In contrast, BERT's kernels have shorter execution times, and throughput improvements are only possible when BERT itself is used as a best-effort task.

Multi-version kernel selection. By leveraging multi-version kernel selection, REEF can achieve an average throughput of $1.18\times$ (up to $1.57\times$). The performance improvement of multi-version kernel selection is more significant when both real-time and best-effort models use fewer CUs. For instance, when using RNET as the real-time task, the average overall throughput increases from $1.16\times$ with DKP to $1.29\times$ with DKP-MV. This improvement is also reflected in the CU usage (shown in Figure 24), which increases from an average of 49% with DKP to 72% with DKP-MV. However, multi-version kernel selection shows limited benefits for models with inherently high CU usage, as their performance is constrained by limited resources rather than kernel execution time. Notably, while DKP results in zero throughput for best-effort tasks in four cases (YOLO-GPT2, BERT-RNET, BERT-VGG, and GPT2-YOLO), DKP-MV successfully prevents best-effort task starvation in these scenarios.

Optimizations. To investigate the impact of optimizations on both performance and memory usage, we first evaluate the overhead using different implementations of the function pointer on the GPU. We measured such overhead by launching real-time kernels through the dkp kernel without padding any best-effort kernels. As shown in Figure 25, the default function pointer implementation (Default) incurs execution time overhead from 97% up to 503% for real-time tasks with different DNN models. By using the global function pointer (GlobalPtr), the overhead is significantly reduced to an average of 38.5% (ranging from 11.5% to 119%), as it eliminates the limit on the number of registers for device function pointers and avoids additional register saving and restoring during the function call. Finally, the overhead drops to 0.8% on average (1.18% at most) by using proxy kernel (ProxyKernel), which can dynamically allocate registers to each kernel and maximize CU occupancy. The minimal overhead comes from the logic branch of CU partition and the initial state preparation for global function pointers.

Kernel selection. Figure 26 shows the average time of kernel selection for DISB A–E during dynamic kernel padding. For workloads with a single BE client (DISB A and B), REEF takes about $0.4\mu s$ to select best-effort kernels for the given real-time kernel. The selection time increases to $1.2\mu s$ for workloads with multiple BE clients (DISB C, D, and E) due to more candidates. In general, the cost of kernel selection is quite trivial and can be easily hidden by kernel execution.

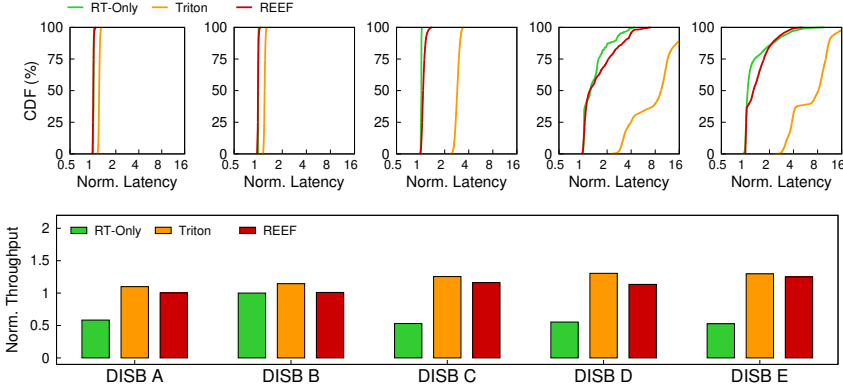


Fig. 28. Comparison of (a) normalized end-to-end latency for RT tasks, and (b) normalized overall throughput of all tasks using Triton [22] and REEF. Both Triton and REEF use the TensorRT [68] to execute DNN models. **Testbed:** One NVIDIA GV100 GPU.

Overhead. To further study the accuracy of kernel selection, we evaluate the execution time overhead for the real-time kernel due to padding best-effort kernels on all DISB workloads. As shown in Figure 27, over 49.1% of real-time kernels are not negatively impacted by concurrent execution with best-effort kernels, and the overhead of more than 90% real-time kernels is still less than 9 μ s. The increase in execution time is mainly due to the contention on GPU memory and shared L2 cache. Notably, while such overhead on a real-time kernel may delay the start of the next one, it does not affect the execution time of subsequent kernels. So if a best-effort kernel runs longer than expected, it will only affect the real-time kernel it is padded with.

6.6 Comparison with State-of-the-Art Systems

We further compare the performance of REEF with two state-of-the-art systems: Triton [22], a production-level inference serving system, and Orion [95], a fine-grained GPU sharing system. Both these systems use inference engines with closed-source GPU kernels, i.e., TensorRT [68] for Triton and PyTorch [77] for Orion. REEF does not support dynamic kernel padding in this comparison, as it requires modifying the source code of GPU kernels, which is not allowed for closed-source inference engines. Fortunately, our reset-based preemption implementation on NVIDIA GPU leverages dynamic binary instrumentation to transform the binary code of the GPU kernel at runtime, thereby successfully working with the closed-source inference engines. We still compare the performance of REEF with Triton and Orion in terms of normalized end-to-end latency for real-time tasks and normalized overall throughput of all tasks using the DISB workloads.

Comparison with Triton. Triton is a production-level inference serving system, and has been widely used in the industry. Triton uses multiple GPU streams to serve multiple clients concurrently. To make a fair comparison, we enable the priority feature of Triton’s TensorRT backend, and set the models used by real-time clients to MAX priority, while the models used by best-effort clients were set to MIN priority. The priority feature of TensorRT backend is implemented by setting the CUDA stream priority. The experimental results are shown in Figure 28.

Across all workloads, REEF consistently achieves lower real-time task latency compared to Triton, ranging from 1.12 $\times$ lower in DISB-A to 5.20 $\times$ lower in DISB-E. Specifically, in DISB-A and B, Triton’s real-time tasks experience relatively moderate interference, with an average latency increase of only 1.21 $\times$. However, in DISB-C, the increased number of best-effort clients leads to a more significant latency increase of 2.8 $\times$ for real-time tasks. In contrast, REEF’s preemptive scheduling approach

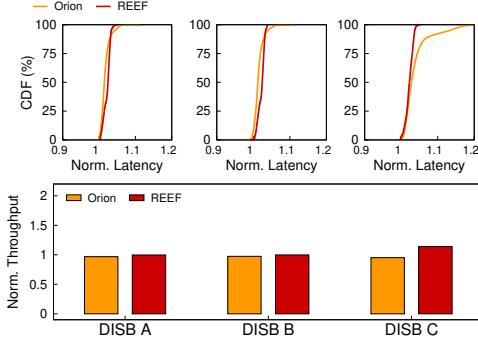


Fig. 29. Comparison of (a) normalized end-to-end latency for RT tasks, and (b) normalized overall throughput of all tasks using Orion [95] and REEF. Both Orion and REEF use the PyTorch [77] to execute DNN models. **Testbed:** One NVIDIA GV100 GPU.

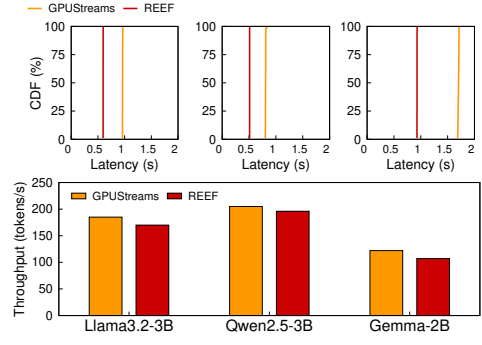


Fig. 30. Comparison of (a) end-to-end latency for RT tasks, and (b) overall throughput using GPUStreams and REEF under DISB-A workload with LLMs. The LLMs are executed using MLC-LLM [64] as the inference engine. **Testbed:** One NVIDIA GV100 GPU.

maintains stable real-time task performance, with a maximum latency increase of only 9% (average 5%) in DISB-C. In terms of throughput, Triton achieves higher average throughput of 1.21 $\times$, while REEF reaches 1.11 $\times$ average throughput.

Comparison with Orion. Orion is a state-of-the-art fine-grained GPU sharing system designed for ML workloads, with the ability to control the interference between co-executing high-priority tasks and low-priority tasks. Because Orion only supports scheduling one high-priority task and multiple low-priority tasks, we compare the performance of REEF with Orion using only DISB A, B and C workloads by setting the real-time client to high-priority and the best-effort clients to low-priority. The experimental results are shown in Figure 29.

In DISB-A and DISB-B, Orion achieves comparable performance to REEF. This is because Orion employs a one-kernel-at-a-time scheduling approach, where it launches only one kernel at a time (prioritizing high-priority kernels), resulting in low latency for real-time tasks. However, this scheduling approach has two significant limitations. First, it cannot preempt executing best-effort kernels, leading to higher tail latency. For instance, Orion’s real-time tasks experience P_{99} latency of 1.05 $\times$ in DISB-A and B, and 1.17 $\times$ in DISB-C, while REEF consistently maintains P_{99} latency below 1.05 $\times$. Second, it incurs high scheduling overhead (similar to having a DQ size of 1), which reduces system throughput. Specifically, Orion achieves only 0.97 $\times$ throughput in DISB-C, while REEF reaches 1.14 $\times$.

6.7 REEF with Large Language Models (LLMs)

REEF is not limited to the models in Table 1. We demonstrate that REEF can support more recent models, such as LLMs. To evaluate REEF’s performance with LLMs, we conduct experiments using the DISB-A workload, where one real-time client and one best-effort client submit inference requests simultaneously. The real-time client submits requests at 50% of the maximum throughput. We test three popular LLM models: Llama3.2-3B [63], Qwen2.5-3B-Instruct [79], and Gemma-2B-IT [33], where both RT and BE clients use the same model in each experiment. We set the batch size to 1 and fix the decode length to 100 tokens per request. All LLM models are executed using MLC-LLM [64] as the inference engine.

Figure 30 shows the end-to-end latency for real-time tasks and the total throughput (tokens/s) for both clients across the three LLM models. The results demonstrate that REEF consistently maintains

compute units (CUs), a separate GPU API to precisely reset CUs is feasible and would be useful to kill and restore all running kernels. Second, we propose a new GPU API that instructs the command processor to discard fetched kernels and stop fetching more kernels from the DQ (DQs). Based on it, DQs can be proactively reset with a hardware-software co-design, replacing our software-only solution (i.e., lazy eviction). Finally, the GPU runtime could provide a high-level API for developers to reset the GPU stream, by discarding kernels buffered in internal data structures (e.g., host queues) and resetting the GPU via two new APIs. We believe that these extensions can greatly simplify implementation, even fully implementing reset-based preemption on closed-source GPUs, and further improve performance, for example instantly preempting the GPU in 10 μ s.

Future low-latency GPU inference systems. For low-latency GPU inference systems, regardless of their scheduling objectives (such as meeting SLOs like Paella [67], or ensuring real-time tasks are not affected like REEF), their core mechanism is how to control interference between concurrent requests, and how to maximize throughput under the premise of controlling interference. REEF proposes reset-based preemption and dynamic kernel padding two techniques that control interference from temporal and spatial dimensions respectively. However, there is still significant room for performance improvement. For example, for dynamic kernel padding, compared to using multiple GPU streams, there is still a performance gap in terms of throughput. Therefore, we believe that future low-latency GPU inference systems need more precise control over GPU computing resource allocation mechanisms, dynamically adjusting each task's resource usage at a finer granularity (even at the nanosecond level). This requires not only new techniques on the scheduling system side, but also new mechanisms on the GPU hardware side.

8 Related Work

DNN inference serving systems. Prior model serving systems [25, 31, 39, 57, 67, 73, 93, 106, 114] mainly focus on meeting **service-level objectives (SLO)**, typically in the tens of milliseconds [26, 41, 110], and improving overall throughput of datacenter applications. Clockwork [35] leverages the latency predictability of DNN inference to achieve low tail latency. It runs inferences sequentially on dedicated GPUs to provide predictable performance. Clipper [24] and Nexus [90] enables batching inferences on the same model to improve GPU utilization and inference throughput. Abacus [26] enables simultaneous DNN inferences by accurately predicting the latency of the overlapped operators. INFaaS [87] can automatically select the right variant with different optimizations for each inference to meet diverse SLOs. AlpaServe [57] leverages model parallelism to reduce the serving latency of bursty requests. Shepherd [114] aggregates inference request streams into moderately-sized groups to improve the predictability of the request arrival pattern. USHER [93] merges multiple models with similar weights to minimize the contention in GPU cache when concurrent inferences are running on the same GPU. Paella [67] uses a similar kernel transformation approach to REEF to bypass the built-in GPU scheduler for low-latency inference. However, the latency SLOs for datacenter applications are much more relaxed than those for real-time systems, for example $2\times$ of their solo-run latencies [26, 93]. Therefore, using non-preemptive scheduling or batching scheme is effective for datacenter applications, but not for real-time scenarios (e.g., autonomous vehicles). Furthermore, the design of REEF is orthogonal to the above distributed serving systems. Two key mechanisms in REEF can also be integrated into them to improve per-GPU throughput and preserve low latency for real-time inferences.

There are also some efforts to achieve low-latency inference under circumstances where the models are not pre-loaded in GPU memory. PipeSwitch [7] pipelines the model loading and the kernel execution, based on the observation that DNN models have a layer-by-layer structure. DeepPlan [47] further leverages the direct-host-access facility provided by commodity GPUs and the GPU-GPU communication to reduce the model loading latency. DeepUM [49] transparently

prefetches the CPU memory based on the historical access pattern of the model. INFless [109] leverages both the long-term and short-term application idle times to decide when to pre-load the models. ServerlessLLM [32] optimizes the structure of the model checkpoint to fully utilize the bandwidth of multi-tier storage. In contrast, REEF assumes that the models are pre-pinned in GPU memory, and is orthogonal to the above efforts. The two mechanisms in REEF can also be integrated into the above systems to improve the performance for the time period when the models are already loaded in GPU memory.

GPU kernel preemption. Prior work has proposed hardware enhancement to support preemptive GPU scheduling [59, 75, 96, 115]. An intuitive solution is to support context switch on GPUs [96]. However, it is far more expensive on GPU than CPU due to the large context (e.g., a large amount of registers). Zhen et al. [59] proposed lightweight context switching to avoid unnecessary register saving. Tanasić et al. [96] extended the hardware to passively preempt a streaming multiprocessor (SM) of GPU by stopping issuing new thread blocks. Chimera [75] proposed SM flushing to instantly preempt an SM when detecting idempotent execution. Differently, our approach retrofits existing hardware mechanism and requires no modification on the GPU to implement instant preemption.

Apart from the hardware enhancement, prior work has also proposed software techniques to support preemptive GPU scheduling [14, 50–53, 89, 104, 122]. Most prior work employs wait-based preemption, albeit through various approaches. For example, GPES [122] slices a GPU kernel into multiple sub-kernels and schedule tasks at the boundary of sub-kernels. EffiSha [14] and FLEP [104] leverage kernel transformation to preempt the running kernel at the end of each block. Lee et al. [53] proposes an idempotence-based preemption mechanism that can instantly preempt the running kernel without waiting for its completion similar to our reset-based preemption mechanism. The preemption mechanism in REEF is different from these work mainly in two aspects. First, REEF not only preempts the currently running kernel but also evicts the kernels in the software queues. This capability is crucial for DNN inference tasks, which often involve numerous asynchronously launched kernels. In contrast, prior work primarily concentrates on scheduling single-kernel applications. Second, while both approaches leverage the idempotence property of GPU kernels, they differ in their handling of non-idempotent kernels. Lee et al. employ a transaction scheme that involves copying the input data of non-idempotent kernels prior to their execution. This method can result in significant performance overhead during normal execution, even when preemption is not invoked. In contrast, REEF utilizes a novel multi-kernel idempotence property to recover non-idempotent kernels, incurring no overhead during normal execution.

GPU multitasking. There have been many efforts to concurrently execute multiple GPU kernels for high throughput [9, 34, 58, 74, 76, 100, 101, 103]. For DNN computation, Rammer [62] takes a holistic approach to exploit both inter- and intra-kernel parallelisms at compile time, which uses static kernel fusion [100] to enforce the CU assignments of the concurrent kernels. However, static kernel fusion requires the fused kernels to be known at compile time, which is not applicable for dynamic task scheduling in REEF. REEF proposes dynamic kernel padding to allow making scheduling decisions at runtime. Prior work has also proposed approaches to model and predict the slowdown of concurrent kernel execution [15, 16, 117, 119]. DASE [44] models the memory contention of concurrent kernels. Themis [118] uses a neural network to predict the performance interference. The prediction can help make scheduling decisions to match the latency requirements of real-time kernels. However, the prediction cannot always be accurate, and the slowdown actually happens. Differently, dynamic kernel padding in REEF enforces concurrent kernels to use only GPU resources leftover from the real-time kernel.

9 Conclusion

This article presented REEF, the first DNN inference serving system for commodity GPUs with microsecond-scale kernel preemption and controlled concurrent execution in GPU scheduling. REEF achieves both low-latency and work-conserving for concurrent real-time and best-effort tasks. First, REEF can launch a real-time kernel on the GPU by proactively killing and restoring best-effort kernels at microsecond-scale. Second, REEF can dynamically pad the real-time kernel with appropriate best-effort kernels to fully exploit the GPU with negligible overhead. In addition, we built a new benchmark (DISB) for DNN inference serving that contains diverse workloads and a real-world trace. Evaluation using DISB and comprehensive microbenchmarks confirmed the efficacy and efficiency of REEF on both AMD and NVIDIA GPUs.

References

- [1] Jacob T. Adriaens, Katherine Compton, Nam Sung Kim, and Michael J. Schulte. 2012. The case for GPGPU spatial multitasking. In *Proceedings of the 2012 IEEE 18th International Symposium on High-Performance Computer Architecture (HPCA'12)*. IEEE Computer Society, USA, 1–12. DOI : <https://doi.org/10.1109/HPCA.2012.6168946>
- [2] Miguel Alcon, Hamid Tabani, Leonidas Kosmidis, Enrico Mezzetti, Jaume Abella, and Francisco J. Cazorla. 2020. Timing of autonomous driving software: problem analysis and prospects for future solutions. In *2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 267–280. DOI : <https://doi.org/10.1109/RTAS48715.2020.000-1>
- [3] AMD ROCm. 2022. AMD ROCm Platform Documentation. Retrieved from <https://rocm-docs.amd.com/>
- [4] Apollo Auto. 2022. Apollo: Architecture/Hardware Connection. Retrieved from <https://github.com/ApolloAuto/apollo>
- [5] Apollo Auto. 2022. Apollo Perception Module. Retrieved from <https://github.com/ApolloAuto/apollo/tree/master/modules/perception>
- [6] Junjie Bai, Fang Lu, Ke Zhang, et al. 2019. ONNX: Open Neural Network Exchange. Retrieved from <https://github.com/onnx/onnx>
- [7] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. 2020. PipeSwitch: Fast pipelined context switching for deep learning applications. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI'20)*. 499–514. Retrieved from <https://www.usenix.org/conference/osdi20/presentation/bai>
- [8] Baidu. 2022. Apollo. Retrieved from <https://apollo.auto/>
- [9] Joshua Bakita and James H. Anderson. 2023. Hardware compute partitioning on NVIDIA GPUs*. *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS) (2023)*, 54–66. Retrieved from <https://api.semanticscholar.org/CorpusID:259235797>
- [10] C. Basaran and K. Kang. 2012. Supporting preemptive task executions and memory copies in GPGPUs. In *24th Euromicro Conference on Real-Time Systems (ECRTS'12)*. 287–296.
- [11] Karsten Behrendt, Libor Novak, and Rami Botros. 2017. A deep learning approach to traffic lights: Detection, tracking, and classification. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*. 1370–1377. DOI : <https://doi.org/10.1109/ICRA.2017.7989163>
- [12] Nicola Capodici, Roberto Cavicchioli, Marko Bertogna, and Aingara Paramakuru. 2018. Deadline-Based Scheduling for GPU with Preemption Support. In *2018 IEEE Real-Time Systems Symposium (RTSS)*. 119–130. DOI : <https://doi.org/10.1109/RTSS.2018.00021>
- [13] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. 2009. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*. 44–54. DOI : <https://doi.org/10.1109/IISWC.2009.5306797>
- [14] Guoyang Chen, Yue Zhao, Xipeng Shen, and Huiyang Zhou. 2017. EffiSha: A software framework for enabling efficient preemptive scheduling of GPU. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'17)*. Association for Computing Machinery, Austin, Texas, USA, 3–16. DOI : <https://doi.org/10.1145/3018743.3018748>
- [15] Quan Chen, Hailong Yang, Minyi Guo, Ram Srivatsa Kannan, Jason Mars, and Lingjia Tang. 2017. Prophet: Precise QoS prediction on non-preemptive accelerators to improve utilization in warehouse-scale computers. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'17)*. Association for Computing Machinery, Xi'an, China, 17–32. DOI : <https://doi.org/10.1145/3037697.3037700>
- [16] Quan Chen, Hailong Yang, Jason Mars, and Lingjia Tang. 2016. Baymax: QoS awareness and increased utilization for non-preemptive accelerators in warehouse scale computers. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'16)*. Association for Computing Machinery, Atlanta, Georgia, USA, 681–696. DOI : <https://doi.org/10.1145/2872362.2872368>

- [17] T. Chen, T. Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, L. Ceze, Carlos Guestrin, and A. Krishnamurthy. 2018. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI'18)*.
- [18] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan M. Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient primitives for deep learning. arXiv:1410.0759. Retrieved from <https://arxiv.org/abs/1410.0759>
- [19] Green Car Congress. 2017. New ultrafast camera for self-driving vehicles and drones. Retrieved from <https://www.greencarcongress.com/2017/02/20170217-ntu.html>
- [20] Brian F. Cooper. 2022. YCSB Core Workloads. Retrieved from <https://github.com/brianfrankcooper/YCSB/wiki/Core-Workloads>
- [21] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *1st ACM Symposium on Cloud Computing (SoCC'10)*. 143–154. DOI: <https://doi.org/10.1145/1807128.1807152>
- [22] NVIDIA Corporation. Triton Inference Server: An Optimized Cloud and Edge Inferencing Solution. Retrieved from <https://github.com/triton-inference-server>. (n. d.).
- [23] Alexander Craik, Yongtian He, and José Luis Contreras-Vidal. 2019. Deep learning for electroencephalogram (EEG) classification tasks: A review. *Journal of Neural Engineering* 16, 3 (June 2019), 031001. DOI: <https://doi.org/10.1088/1741-2552/ab0ab5>
- [24] D. Crankshaw, Xin Wang, Giulio Zhou, M. Franklin, Joseph E. Gonzalez, and I. Stoica. 2017. Clipper: A low-latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI'17)*.
- [25] Weihao Cui, Mengze Wei, Quan Chen, Xiaoxin Tang, Jingwen Leng, Li Li, and Mingyi Guo. 2019. Ebird: Elastic batch for improving responsiveness and throughput of deep learning services. In *2019 IEEE 37th International Conference on Computer Design (ICCD)*. 497–505. DOI: <https://doi.org/10.1109/ICCD46524.2019.00075>
- [26] Weihao Cui, Han Zhao, Quan Chen, Ningxin Zheng, Jingwen Leng, Jieru Zhao, Zhuo Song, Tao Ma, Yong Yang, Chao Li, and Minyi Guo. 2021. Enable simultaneous DNN services based on deterministic operator overlap and precise latency prediction. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'21)*. Association for Computing Machinery, St. Louis, Missouri. DOI: <https://doi.org/10.1145/3458817.3476143>
- [27] Marc A. de Kruijf, Karthikeyan Sankaralingam, and Somesh Jha. 2012. Static analysis and compiler design for idempotent processing. In *33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'12)*. 475–486. DOI: <https://doi.org/10.1145/2254064.2254120>
- [28] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805. Retrieved from <https://arxiv.org/abs/1810.04805>
- [29] ROCm documentation. 2022. GCN Native ISA LLVM Code Generator: Kernel Dispatch. Retrieved from https://rocmdocs.amd.com/en/latest/ROCm_Compiler_SDK/ROCm-Native-ISA.html
- [30] Andre Esteva, Alexandre Robicquet, Bharath Ramsundar, Volodymyr Kuleshov, Mark DePristo, Katherine Chou, Claire Cui, Greg Corrado, Sebastian Thrun, and Jeff Dean. 2019. A guide to deep learning in healthcare. *Nature medicine* 25, 1 (2019), 24–29.
- [31] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. 2021. TurboTransformers: an efficient GPU serving system for transformer models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'21)*. Association for Computing Machinery, Virtual Event, Republic of Korea, 389–402. DOI: <https://doi.org/10.1145/3437801.3441578>
- [32] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. {ServerlessLLM}:{Low-Latency} serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI'24)*. 135–153.
- [33] Google. 2024. Gemma-2B-IT. Retrieved from <https://huggingface.co/google/gemma-2b-it>
- [34] Chris Gregg, Jonathan Dorn, K. Hazelwood, and K. Skadron. 2012. Fine-grained resource sharing for concurrent GPGPU kernels. In *4th USENIX Workshop on Hot Topics in Parallelism (HotPar'12)*.
- [35] A. Gujarati, Reza Karimi, Safya Alzayat, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI'20)*.
- [36] Mingcong Han, Weihang Shen Rong Chen, Binyu Zang, and Haibo Chen. 2025. Holistic Heterogeneous Scheduling for Autonomous Applications using Fine-grained, Multi-XPU Abstraction. (2025). arXiv:2508.09503. Retrieved from <https://arxiv.org/abs/2508.09503>
- [37] Mingcong Han, Weihang Shen, Guanwen Peng, Rong Chen, and Haibo Chen. 2024. Microsecond-scale Dynamic Validation of Idempotency for GPU Kernels. (2024). arXiv:2410.23661. Retrieved from <https://arxiv.org/abs/2410.23661>

- [38] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 539–558. Retrieved from <https://www.usenix.org/conference/osdi22/presentation/han>
- [39] Johann Hauswald, Yiping Kang, Michael A. Laurenzano, Quan Chen, Cheng Li, Trevor Mudge, Ronald G. Dreslinski, Jason Mars, and Lingjia Tang. 2015. DjiNN and Tonic: DNN as a service and its implications for future warehouse scale computers. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA'15)*, Association for Computing Machinery, Portland, Oregon, 27–40. DOI : <https://doi.org/10.1145/2749469.2749472>
- [40] Kaiming He, X. Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (CVPR'16)*. 770–778. DOI : <https://doi.org/10.1109/CVPR.2016.90>
- [41] Jeremy Hermann and Mike Del Balso. 2017. Meet Michelangelo: Uber's Machine Learning Platform. Retrieved from <https://eng.uber.com/michelangelo-machine-learning-platform/>
- [42] Geoffrey Hinton, Li Deng, Dong Yu, George E Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N. Sainath, et al. 2012. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine* 29, 6 (2012), 82–97.
- [43] Connor Holmes, Daniel Mawhirter, Yuxiong He, Feng Yan, and Bo Wu. 2019. GRNN: Low-latency and scalable RNN inference on GPUs. In *Proceedings of the Fourteenth EuroSys Conference 2019 (EuroSys'19)*. Association for Computing Machinery, Dresden, Germany. DOI : <https://doi.org/10.1145/3302424.3303949>
- [44] Qingda Hu, Jiwu Shu, Jie Fan, and Youyou Lu. 2016. Run-time performance estimation and fairness-oriented scheduling policy for concurrent GPGPU applications. In *2016 45th International Conference on Parallel Processing (ICPP)*. 57–66. DOI : <https://doi.org/10.1109/ICPP.2016.14>
- [45] Saksham Jain, Iljoo Baek, Shige Wang, and Ragunathan Rajkumar. 2019. Fractional GPUs: Software-based compute and memory bandwidth reservation for GPUs. In *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 29–41. DOI : <https://doi.org/10.1109/RTAS.2019.00011>
- [46] Wonseok Jang, Hansaem Jeong, Kyungtae Kang, Nikil Dutt, and Jong-Chan Kim. 2020. R-TOD: Real-time object detector with minimized end-to-end delay for autonomous driving. In *2020 IEEE Real-Time Systems Symposium (RTSS)*. 191–204. DOI : <https://doi.org/10.1109/RTSS49844.2020.00027>
- [47] Jinwoo Jeong, Seungsu Baek, and Jeongseob Ahn. 2023. Fast and efficient model serving using multi-GPUs with direct-host-access. In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys'23)*. Association for Computing Machinery, New York, NY, USA, 249–265. DOI : <https://doi.org/10.1145/3552326.3567508>
- [48] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'19)*. Association for Computing Machinery, Huntsville, Ontario, Canada, 47–62. DOI : <https://doi.org/10.1145/3341301.3359630>
- [49] Jaehoon Jung, Jinpyo Kim, and Jaejin Lee. 2023. DeepUM: Tensor migration and prefetching in unified memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 207–221. DOI : <https://doi.org/10.1145/3575693.3575736>
- [50] Shinpei Kato, Karthik Lakshmanan, Aman Kumar, Mihir Kelkar, Yutaka Ishikawa, and Ragunathan Raj Rajkumar. 2011. RGEM: A Responsive GPGPU Execution Model for Runtime Engines. *2011 IEEE 32nd Real-Time Systems Symposium* (2011), 57–66. Retrieved from <https://api.semanticscholar.org/CorpusID:8258940>
- [51] Shinpei Kato, Karthik Lakshmanan, Ragunathan Raj Rajkumar, and Yutaka Ishikawa. 2011. TimeGraph: GPU scheduling for real-time multi-tasking environments. In *USENIX Annual Technical Conference*. Retrieved from <https://api.semanticscholar.org/CorpusID:18344830>
- [52] Shinpei Kato, Michael McThrow, Carlos Maltzahn, and Scott A. Brandt. 2012. Gdev: First-class GPU resource management in the operating system. In *USENIX Annual Technical Conference*. Retrieved from <https://api.semanticscholar.org/CorpusID:3200101>
- [53] Hyeonsu Lee, Hyunjun Kim, Cheolgi Kim, Hwansoo Han, and Euseong Seo. 2021. Idempotence-based preemptive GPU kernel scheduling for embedded systems. *IEEE Transactions on Computers* 70, 3 (2021), 332–346. DOI : <https://doi.org/10.1109/TC.2020.2988251>
- [54] TIMOTHY B. LEE. 2019. Tesla's autonomy event: Impressive progress with an unrealistic timeline. Retrieved from <https://arstechnica.com/cars/2019/04/teslas-autonomy-event-impressive-progress-with-an-unrealistic-timeline/>
- [55] Jingwen Leng, Alper Buyuktosunoglu, Ramon Bertran Monfort, Pradip Bose, Quan Chen, Minyi Guo, and Vijay Janapa Reddi. 2020. Asymmetric resilience: Exploiting task-level idempotency for transient error recovery in accelerator-based systems. *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA) (2020)*, 44–57.

- [56] LG Electronics Inc. 2022. Running Apollo 5.0 with SVL Simulator. Retrieved from <https://www.svl simulator.com/docs/system-under-test/apollo5-0-instructions/>
- [57] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, et al. 2023. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI'23)*. USENIX Association, Boston, MA, 663–679. Retrieved from <https://www.usenix.org/conference/osdi23/presentation/li-zhouhan>
- [58] Yun Liang, Huynh Phung Huynh, Kyle Rupnow, Rick Siow Mong Goh, and Deming Chen. 2015. Efficient GPU spatial-temporal multitasking. *IEEE Trans. Parallel Distrib. Syst.* 26, 3 (March 2015), 748–760. DOI : <https://doi.org/10.1109/TPDS.2014.2313342>
- [59] Zhen Lin, Lars Nyland, and Huiyang Zhou. 2016. Enabling efficient preemption for SIMT architectures with lightweight context switching. In *SC'16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 898–908. DOI : <https://doi.org/10.1109/SC.2016.76>
- [60] LLVM. 2021. User Guide for AMDGPU Backend. Retrieved from <https://llvm.org/docs/AMDGPUUsage.html>
- [61] Justin Luitjens. CUDA Streams—Best Practices and Common Pitfalls. Retrieved from <http://on-demand.gputechconf.com/gtc/2014/presentations/S4158-cuda-streams-best-practices-common-pitfalls.pdf>. (n. d.).
- [62] Lingxiao Ma, Z. Xie, Zhi Yang, J. Xue, Youshan Miao, Wei Cui, W. Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling holistic deep learning compiler optimizations with rTasks. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI'20)*. 881–897.
- [63] Meta AI. 2024. Llama-3.2-3B. Retrieved from <https://huggingface.co/meta-llama/Llama-3.2-3B>. (2024). Hugging Face Model.
- [64] MLC team. 2023-2025. MLC-LLM. (2023-2025). Retrieved from <https://github.com/mlc-ai/mlc-llm>
- [65] Pavlo Molchanov, Shalini Gupta, Kihwan Kim, and Kari Pulli. 2015. Multi-sensor System for driver's hand-gesture recognition. *11th IEEE International Conference and Workshops on Automatic Face and Gesture Recognition 1* (2015), 1–8.
- [66] Deepak Narayanan, Keshav Santhanam, Amar Phanishayee, and Matei Zaharia. 2018. Accelerating deep learning workloads through efficient multi-model execution. In *NeurIPS Workshop on Systems for Machine Learning*. 20.
- [67] Kelvin K. W. Ng, Henri Maxime Demoulin, and Vincent Liu. 2023. Paella: Low-latency model serving with software-defined GPU scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP'23)*. Association for Computing Machinery, New York, NY, USA, 595–610. DOI : <https://doi.org/10.1145/3600006.3613163>
- [68] NVIDIA. NVIDIA TensorRT. Retrieved from <https://developer.nvidia.com/tensorrt>. (n. d.).
- [69] NVIDIA. NVIDIA/open-gpu-kernel-modules: NVIDIA Linux open GPU kernel module source. <https://github.com/NVIDIA/open-gpu-kernel-modules>. (n. d.). Retrieved from <https://github.com/NVIDIA/open-gpu-kernel-modules>
- [70] NVIDIA. 2016. NVIDIA Tesla P100. Retrieved from <http://www.nvidia.com/object/pascal-architecture-whitepaper.html>
- [71] NVIDIA. 2021. CUDA Toolkit: Develop, Optimize and Deploy GPU-Accelerated Apps. Retrieved from <https://developer.nvidia.com/cuda-toolkit>
- [72] NVIDIA. 2024. Blackwell Instruction Set. Retrieved from <https://docs.nvidia.com/cuda/cuda-binary-utilities/index.html#blackwell-blackwell-instruction-set>
- [73] Christopher Olston, Fangwei Li, Jeremiah Harmsen, Jordan Soyke, Kiril Gorovoy, Li Lao, Noah Fiedel, Sukriti Ramesh, and Vinu Rajashekhar. 2017. TensorFlow-serving: Flexible, high-performance ML serving. In *Workshop on ML Systems at NIPS 2017*.
- [74] S. Pai, M. J. Thazhuthaveetil, and R. Govindarajan. 2013. Improving GPGPU concurrency with elastic kernels. In *Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'13)*.
- [75] Jason Jong Kyu Park, Yongjun Park, and Scott Mahlke. 2015. Chimera: Collaborative preemption for multitasking on a shared GPU. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'15)*. Association for Computing Machinery, Istanbul, Turkey, 593–606. DOI : <https://doi.org/10.1145/2694344.2694346>
- [76] Jason Jong Kyu Park, Yongjun Park, and Scott Mahlke. 2017. Dynamic resource management for efficient utilization of multitasking GPUs. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'17)*. Association for Computing Machinery, Xi'an, China, 527–540. DOI : <https://doi.org/10.1145/3037697.3037707>
- [77] Adam Paszke, S. Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, N. Gimeshain, L. Antiga, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *33rd International Conference on Neural Information Processing Systems (NeurIPS'19)*.
- [78] Reid Pinkham, Andrew Berkovich, and Zhengya Zhang. 2021. Near-sensor distributed DNN processing for augmented and virtual reality. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 11, 4 (2021), 663–676. DOI : <https://doi.org/10.1109/JETCAS.2021.3121259>

- [79] Qwen Team. 2024. Qwen2.5-3B-Instruct. Retrieved from <https://huggingface.co/Qwen/Qwen2.5-3B-Instruct>. (2024). Hugging Face Model.
- [80] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. OpenAI.
- [81] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 779–788. DOI: <https://doi.org/10.1109/CVPR.2016.91>
- [82] Joseph Redmon and Ali Farhadi. 2018. YOLOv3: An Incremental Improvement. (2018). Retrieved from <http://arxiv.org/abs/1804.02767> cite arxiv:1804.02767Comment: Tech Report.
- [83] Steve Rennich. CUDA C/C++ Streams and Concurrency. Retrieved from <https://developer.download.nvidia.cn/CUDA/training/StreamsAndConcurrencyWebinar.pdf>. (n. d.).
- [84] ROCm Core Technology. 2022. AMD GPU kernel driver with KFD. Retrieved from <https://github.com/RadeonOpenCompute/ROCK-Kernel-Driver>
- [85] ROCm Core Technology. 2022. AMD GPU kernel driver with KFD: unmap_queues_cpsch. Retrieved from https://github.com/RadeonOpenCompute/ROCK-Kernel-Driver/blob/master/drivers/gpu/drm/amd/amdkfd/kfd_device_queue_manager.c
- [86] ROCm Developer Tools. 2022. HIP: C++ Heterogeneous-Compute Interface for Portability. Retrieved from <https://github.com/ROCm-Developer-Tools/HIP>
- [87] Francisco Romero, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2021. INFaaS: Automated model-less inference serving. In *USENIX Annual Technical Conference (ATC'21)*. 397–411.
- [88] Guodong Rong, Byung Hyun Shin, Hadi Tabatabaee, Qiang Lu, Steve Lemke, Márton Mozeiko, Eric Boise, Geehoon Uhm, Mark Gerow, Shalin Mehta, et al. 2020. LGSVL Simulator: A High Fidelity Simulator for Autonomous Driving. In *IEEE 23rd International Conference on Intelligent Transportation Systems Conference (ITSC'20)*. 1–6. DOI: <https://doi.org/10.1109/ITSC45102.2020.9294422>
- [89] Christopher J. Rossbach, Jon Currey, Mark Silberstein, Baishakhi Ray, and Emmett Witchel. 2011. PTask: Operating system abstractions to manage GPUs as compute devices. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles (SOSP'11)*. Association for Computing Machinery, New York, NY, USA, 233–248. DOI: <https://doi.org/10.1145/2043556.2043579>
- [90] Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. 2019. Nexus: A GPU cluster engine for accelerating DNN-based video analysis. *27th ACM Symposium on Operating Systems Principles* (2019).
- [91] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. 2025. XSched: Preemptive scheduling for diverse XPU. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 671–692. Retrieved from <https://www.usenix.org/conference/osdi25/presentation/shen-weihang>
- [92] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. 2023. Welder: Scheduling deep learning memory access via tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 701–718.
- [93] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. 2024. USHER: Holistic interference avoidance for resource optimized ML inference. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 947–964. Retrieved from <https://www.usenix.org/conference/osdi24/presentation/shubha>
- [94] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. arXiv:1409.1556. Retrieved from <https://arxiv.org/abs/1409.1556>
- [95] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, fine-grained GPU sharing for ML applications. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys'24)*. Association for Computing Machinery, New York, NY, USA, 1075–1092. DOI: <https://doi.org/10.1145/3627703.3629578>
- [96] I. Tanasić, Isaac Gelado, Javier Cabezas, A. Ramírez, N. Navarro, and M. Valero. 2014. Enabling preemptive multiprogramming on GPUs. In *Proceeding of the 41st Annual International Symposium on Computer Architecture (ISCA'14)*. IEEE Press, Minneapolis, Minnesota, USA, 193–204.
- [97] TESLARATI. 2021. AMD confirms Tesla's new Model S and Model X will boast RDNA 2 GPUs. Retrieved from <https://www.teslarati.com/tesla-model-s-model-x-mcu3-specs-amd-gpu-confirmed-video/>
- [98] Apache TVM. 2021. Apache TVM: An End to End Machine Learning Compiler Framework for CPUs, GPUs and accelerators. Retrieved from <https://tvm.apache.org/>
- [99] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W. Keckler. 2019. NVBit: A dynamic binary instrumentation framework for NVIDIA GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-52)*. Association for Computing Machinery, Columbus, OH, USA, 372–383. DOI: <https://doi.org/10.1145/3352460.3358307>

- [100] Guibin Wang, YiSong Lin, and Wei Yi. 2010. Kernel Fusion: An effective method for better power efficiency on multithreaded GPU. In *Proceedings of the 2010 IEEE/ACM Int'l Conference on Green Computing and Communications & Int'l Conference on Cyber, Physical and Social Computing (GREENCOM-CPSCOM'10)*. IEEE Computer Society, USA, 344–350. DOI : <https://doi.org/10.1109/GreenCom-CPSCOM.2010.102>
- [101] Jiali Wang, Yankui Wang, Mingcong Han, and Rong Chen. 2025. Colocating ML inference and training with fast GPU memory handover. In *USENIX Annual Technical Conference (ATC'25)*.
- [102] Yuxuan Wang, R. J. Skerry-Ryan, Daisy Stanton, Yonghui Wu, Ron J. Weiss, Navdeep Jaitly, Zongheng Yang, Ying Xiao, Zhifeng Chen, Samy Bengio, et al. 2017. Tacotron: A fully end-to-end text-to-speech synthesis model. arXiv:1703.10135. Retrieved from <https://arxiv.org/abs/1703.10135>
- [103] Zhenning Wang, Jun Yang, Rami Melhem, Bruce Childers, Youtao Zhang, and Minyi Guo. 2016. Simultaneous Multikernel GPU: Multi-tasking throughput processors via fine-grained sharing. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 358–369. DOI : <https://doi.org/10.1109/HPCA.2016.7446078>
- [104] Bo Wu, Xu Liu, Xiaobo Zhou, and Changjun Jiang. 2017. FLEP: Enabling flexible and efficient preemption on GPUs. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'17)*. Association for Computing Machinery, Xi'an, China, 483–496. DOI : <https://doi.org/10.1145/3037697.3037742>
- [105] Yecheng Xiang and Hyoseung Kim. 2019. Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference. In *2019 IEEE Real-Time Systems Symposium (RTSS)*. 392–405. DOI : <https://doi.org/10.1109/RTSS46320.2019.00042>
- [106] Feng Yan, Yuxiong He, Olatunji Ruwase, and Evgenia Smirni. 2018. Efficient deep neural network serving: Fast and furious. *IEEE Transactions on Network and Service Management* 15, 1 (2018), 112–126. DOI : <https://doi.org/10.1109/TNSM.2018.2808352>
- [107] Jinrong Yang, Zimeng Wang, Rong Chen, and Haibo Chen. 2025. A system-level abstraction and service for flourishing AI-powered applications. In *Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys'25)*. Association for Computing Machinery, New York, NY, USA, 9. DOI : <https://doi.org/10.1145/3725783.3764406>
- [108] Ming Yang, Shige Wang, Joshua Bakita, Thanh Vu, F. Donelson Smith, James H. Anderson, and Jan-Michael Frahm. 2019. Re-thinking CNN frameworks for time-sensitive autonomous-driving applications: Addressing an industrial challenge. In *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 305–317. DOI : <https://doi.org/10.1109/RTAS.2019.00033>
- [109] Yanan Yang, Laiping Zhao, Yiming Li, Huanyu Zhang, Jie Li, Mingyang Zhao, Xingzhen Chen, and Keqiu Li. 2022. INFless: A native serverless system for low-latency, high-throughput inference. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 768–781.
- [110] Wai Chee Yau. 2017. How Zendesk Serves TensorFlow Models in Production. Retrieved from <https://zendesk.engineering/how-zendesk-serves-tensorflow-models-in-production-751ee22f0f4b>
- [111] Tsung Tai Yeh, Matthew D. Sinclair, Bradford M. Beckmann, and Timothy G. Rogers. 2021. Deadline-aware offloading for high-throughput accelerators. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 479–492. DOI : <https://doi.org/10.1109/HPCA51647.2021.00048>
- [112] Juheon Yi and Youngki Lee. 2020. Heimdall: mobile GPU coordination platform for augmented reality applications. In *26th Annual International Conference on Mobile Computing and Networking (MobiCom'20)*. 1–14.
- [113] Sebastian Zepf, Javier Hernandez, Alexander Schmitt, Wolfgang Minker, and Rosalind W. Picard. 2020. Driver emotion recognition for intelligent vehicles: A survey. *ACM Comput. Surv.* 53, 3 (July 2020). DOI : <https://doi.org/10.1145/3388790>
- [114] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. 2023. SHEPHERD: Serving DNNs in the Wild. In *Symposium on Networked Systems Design and Implementation*. Retrieved from <https://api.semanticscholar.org/CorpusID:258559393>
- [115] Chen Zhao, Wu Gao, Feiping Nie, and Huiyang Zhou. 2022. A survey of GPU multitasking methods supported by hardware architecture. *IEEE Transactions on Parallel and Distributed Systems* 33 (2022), 1451–1463. Retrieved from <https://api.semanticscholar.org/CorpusID:239197299>
- [116] Hengyu Zhao, Yubo Zhang, Pingfan Meng, Hui Shi, Erran L. Li, Tiancheng Lou, and Jishen Zhao. 2019. Towards safety-aware computing system design in autonomous vehicles. arXiv:1905.08453. Retrieved from <https://arxiv.org/abs/1905.08453>
- [117] Wenyi Zhao, Quan Chen, and Minyi Guo. 2018. KSM: Online application-level performance slowdown prediction for spatial multitasking GPGPU. *IEEE Comput. Archit. Lett.* 17, 2 (July 2018), 187–191. DOI : <https://doi.org/10.1109/LCA.2018.2851207>
- [118] Wenyi Zhao, Quan Chen, Hao Lin, Jianfeng Zhang, Jingwen Leng, Chao Li, Wenli Zheng, Li Li, and Minyi Guo. 2019. Themis: Predicting and reining in application-level slowdown on spatial multitasking GPUs. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 653–663. DOI : <https://doi.org/10.1109/IPDPS.2019.00074>

- [119] Xia Zhao, Magnus Jahre, and Lieven Eeckhout. 2020. HSM: A hybrid slowdown model for multitasking GPUs. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*. Association for Computing Machinery, Lausanne, Switzerland, 1371–1385. DOI: <https://doi.org/10.1145/3373376.3378457>
- [120] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. 2020. Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI'20)*.
- [121] Husheng Zhou, Soroush Bateni, and Cong Liu. 2018. S3DNN: Supervised streaming and scheduling for GPU-accelerated real-time DNN workloads. *2018 IEEE Real-Time and Embedded Technology and Applications Symposium (2018)*, 190–201.
- [122] Husheng Zhou, Guangmo Tong, and Cong Liu. 2015. GPES: A preemptive execution system for GPGPU computing. In *21st IEEE Real-Time and Embedded Technology and Applications Symposium*. 87–97. DOI: <https://doi.org/10.1109/RTAS.2015.7108420>
- [123] Hongyu Zhu, Ruofan Wu, Yijia Diao, Shanbin Ke, Haoyu Li, Chen Zhang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Wei Cui, et al. 2022. {ROLLER}: Fast and efficient tensor compilation for deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 233–248.

Received 1 November 2024; revised 30 May 2025; accepted 5 September 2025